

Backpropagation and Computational Graphs

Scaler School of Technology | Deep Learning I | Week 2 — Session 2/3 | Post-Lecture Review & Practice



Key Takeaways

1 The chain rule decomposes gradients into local products

If L depends on w through $z, h, \hat{y}$: $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h} \cdot \frac{\partial h}{\partial z} \cdot \frac{\partial z}{\partial w}$. Each factor is a simple local gradient. The hard global problem becomes many easy local ones.

2 Computational graphs make backprop systematic

Any computation as a DAG. Forward pass: evaluate left to right, store intermediates. Backward pass: propagate gradients right to left, multiply upstream $\times$ local at each node. Each node is independent — this is why backprop scales to billions of parameters.

3 Backprop = forward + loss + backward + update

Step 1: Forward pass (compute $\hat{y}$). Step 2: Compute loss L . Step 3: Backward pass (compute $\partial L / \partial w$ for every weight). Step 4: Update $w_{\text{new}} = w_{\text{old}} - \eta \cdot \partial L / \partial w$. Repeat. Same loop for **all** architectures.

4 Gradient flow determines what's trainable

Sigmoid max gradient: 0.25. After 10 layers: $0.25^{10} \approx 10^{-6}$ — gradients vanish, early layers can't learn. ReLU gradient = 1 for positive inputs: $1^{10} = 1$ — gradients flow unchanged. ReLU enabled deep learning. PyTorch autograd automates the entire backward pass.

Test Your Understanding

Try answering without looking at slides. Answers are on page 3.

- Q1.** In a computational graph for $f = a \cdot b$, what is $\frac{\partial f}{\partial a}$? Why does this mean we need forward pass values for the backward pass?
- Q2.** A sigmoid network has 20 hidden layers. Estimate the gradient that reaches the first layer. What's the practical consequence?
- Q3.** Our 2-2-1 MLP computed $\hat{y} = 0.594$ with target $y = 1.0$. All gradients were negative. Explain intuitively why this makes sense.
- Q4.** What are the four steps of the backpropagation algorithm, in order?
- Q5.** In PyTorch, what does `loss.backward()` compute, and where are results stored?

Work through these in a Colab notebook. No starter code — figure out the approach from what you learned.

Exercise 1: Verify the Chain Rule with PyTorch

Implement $f(x) = \sin(x^2)$ in PyTorch. Create x with `requires_grad=True`. Compute f , call `backward()`, and read `x.grad`. Then compute the derivative by hand using the chain rule: $f'(x) = 2x \cos(x^2)$. Verify they match at $x = 1.0$, $x = 0.5$, and $x = -2.0$.

Exercise 2: Build a Computational Graph Engine

Create a `Value` class in Python that stores a number and supports `+`, `*`, and `relu()`. Each operation should store the inputs and a `_backward` function. Implement a `backward()` method that traverses the graph in reverse topological order and accumulates gradients. Test on $f(x, y) = (x + y) \cdot (x - y)$ and verify $\partial f / \partial x = 6$, $\partial f / \partial y = -2$ when $x = 3, y = 1$.

Exercise 3: Manual Backprop on a 2-2-1 MLP

Implement the 2-2-1 MLP from the lecture in NumPy (no torch). Use the exact weights from the session: $W_1 = \begin{bmatrix} 0.4 & 0.3 \\ 0.2 & 0.7 \end{bmatrix}$, $b_1 = [0.1, 0.05]$, $W_2 = [0.6, 0.5]$, $b_2 = -0.3$. Input $\mathbf{x} = [0.5, 0.8]$, target $y = 1.0$. Compute: (a) forward pass, (b) MSE loss, (c) all gradients by hand, (d) one weight update with $\eta = 0.1$. Then verify all gradients against PyTorch autograd.

Challenge Problem

Train the MLP to Convergence

Take the 2-2-1 MLP from Exercise 3 and run the full training loop (forward $\rightarrow$ loss $\rightarrow$ backward $\rightarrow$ update) for 1000 iterations with $\eta = 0.5$. Plot the loss curve. How many iterations until $\hat{y} > 0.99$? Then try with sigmoid hidden activations instead of ReLU — does it converge faster or slower? Why?

- **Rumelhart, Hinton & Williams (1986) — Learning Representations by Back-Propagating Errors** — Nature, Vol. 323
The paper that launched modern neural networks. Demonstrated that backprop can train multi-layer networks.
- **Goodfellow et al. — Deep Learning, Ch. 6.5** — deeplearningbook.org
Formal treatment of backpropagation and computational graphs.
- **Karpathy — micrograd** — github.com/karpathy/micrograd
A tiny autograd engine in 100 lines of Python. Builds exactly the Value class from Exercise 2.
- **3Blue1Brown — Backpropagation Calculus** — YouTube, Chapter 4
Beautiful visual explanation of the chain rule applied to neural networks.

Online Resources & Tools

- **PyTorch Autograd tutorial** — pytorch.org/tutorials/beginner/autograd_tutorial
- **TensorFlow Playground** — toggle “show gradients” to see gradient flow
- **CS231n Backprop Notes** — Stanford’s excellent derivation and examples
- **Desmos** — plot sigmoid and ReLU derivatives interactively
- **Karpathy micrograd video** — YouTube, 2.5 hours building autograd from scratch
- **Google Colab** — free GPU for all exercises

Fill the Gaps — Prerequisite Refreshers

Felt lost on a specific concept? These resources will help.

- **Chain rule** — Khan Academy: “Chain rule” in the derivatives section
- **Partial derivatives** — Khan Academy: “Multivariable calculus — partial derivatives”
- **Matrix calculus** — The Matrix Cookbook (free PDF) for derivative of matrix products
- **The forward pass (Session 2)** — Review your Session 2 notes or the notebook

Answers — Test Your Understanding

- A1.** $\frac{\partial f}{\partial a} = b$ (the other input). The backward pass needs b ’s value, which was computed during the forward pass. Without stored forward values, we can’t compute gradients.
- A2.** $0.25^{20} \approx 10^{-12}$. The gradient is essentially zero. The first layer cannot learn — its weights stay at their random initialization. This is the vanishing gradient problem.
- A3.** The output 0.594 is below the target 1.0 — the network is “too low.” Negative gradients mean $\partial L / \partial w < 0$: increasing w decreases the loss. So gradient descent increases the weights, pushing the output up toward the target.
- A4.** (1) Forward pass — compute prediction. (2) Compute loss. (3) Backward pass — compute gradients for every weight. (4) Update weights: $w_{\text{new}} = w_{\text{old}} - \eta \cdot \partial L / \partial w$. Repeat.
- A5.** It computes $\partial L / \partial w$ for every parameter with `requires_grad=True`, by traversing the computational graph in reverse. Results are stored in each tensor’s `.grad` attribute.